

TP3

Créer un projet JAR / dépendance Spring Web

localhost:8080

The screenshot displays an IDE interface with a project named 'spring1' and a class named 'Monapi.java'. The code in 'Monapi.java' defines two REST endpoints: 'bonjour()' and 'bonsoir()'. The console output shows the application starting successfully on port 8080. Below the IDE, a browser window is shown with the address 'localhost:8080/b' and the response 'bonjour'.

```
package com.example.spring1;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class Monapi {
    @GetMapping(value="/b")
    public String bonjour() {
        return "bonjour";
    }

    @GetMapping(value="/bn")
    public String bonsoir() {
        return "Bonsoir";
    }
}
```

Run: Spring1Application (v3.4.3)

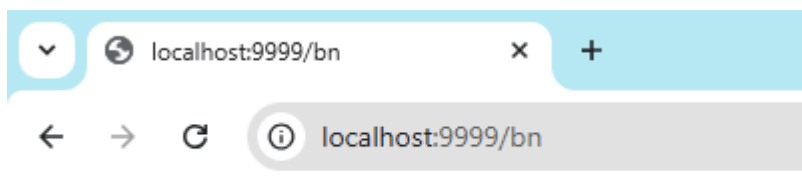
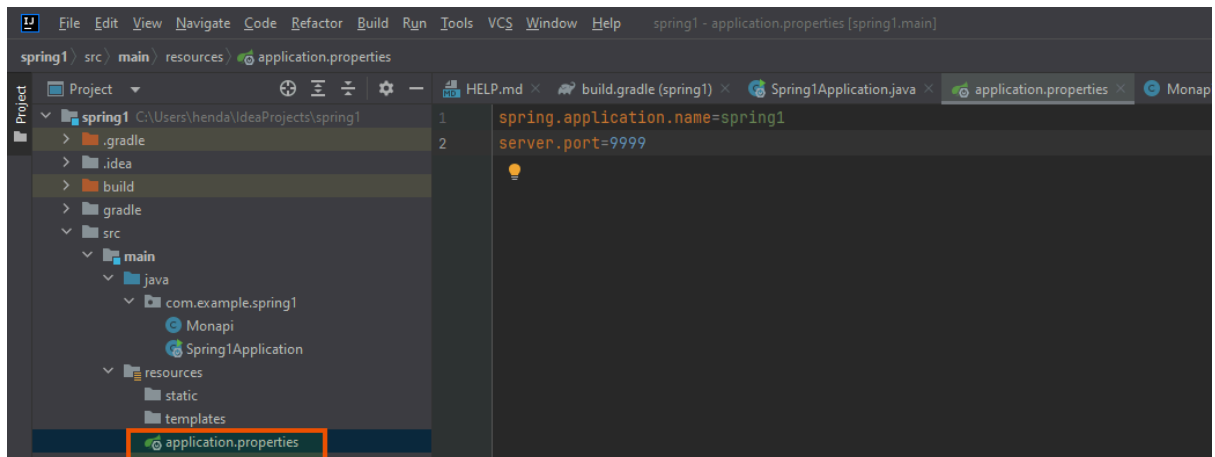
```
2025-03-16T15:50:29.985+01:00 INFO 23604 --- [spring1] [main] com.example.spring1.Spring1Application : Starting Spring1Application using Java 17.0.12 with PID 23604
2025-03-16T15:50:29.989+01:00 INFO 23604 --- [spring1] [main] com.example.spring1.Spring1Application : No active profile set, falling back to 1 default profile:
2025-03-16T15:50:30.665+01:00 INFO 23604 --- [spring1] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2025-03-16T15:50:30.675+01:00 INFO 23604 --- [spring1] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-03-16T15:50:30.675+01:00 INFO 23604 --- [spring1] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.36]
2025-03-16T15:50:30.713+01:00 INFO 23604 --- [spring1] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-03-16T15:50:30.714+01:00 INFO 23604 --- [spring1] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 6
2025-03-16T15:50:31.038+01:00 INFO 23604 --- [spring1] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-03-16T15:50:31.045+01:00 INFO 23604 --- [spring1] [main] com.example.spring1.Spring1Application : Started Spring1Application in 1.767 seconds (process runn
2025-03-16T15:57:41.952+01:00 INFO 23604 --- [spring1] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
```

localhost:8080/b

← → ↻ ⓘ localhost:8080/b

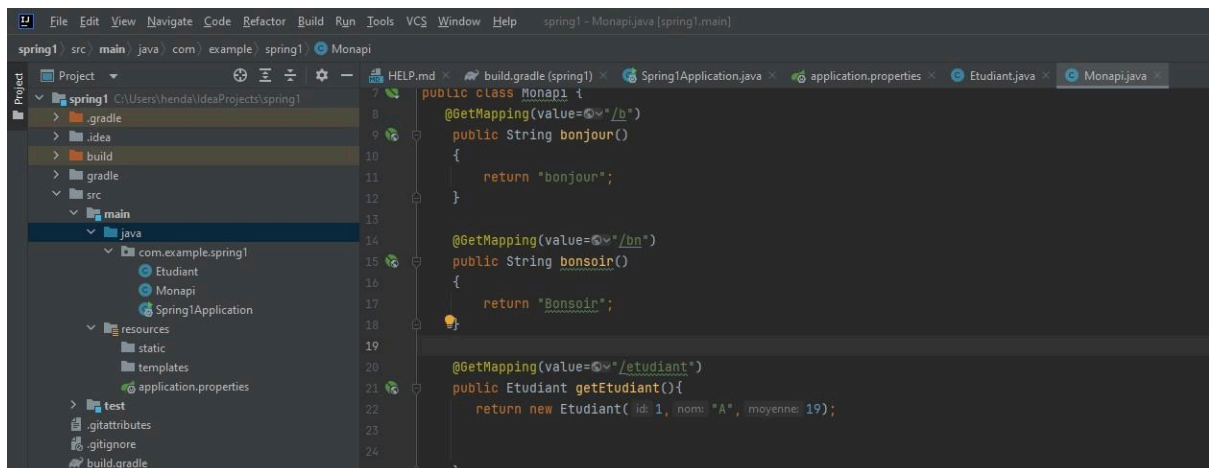
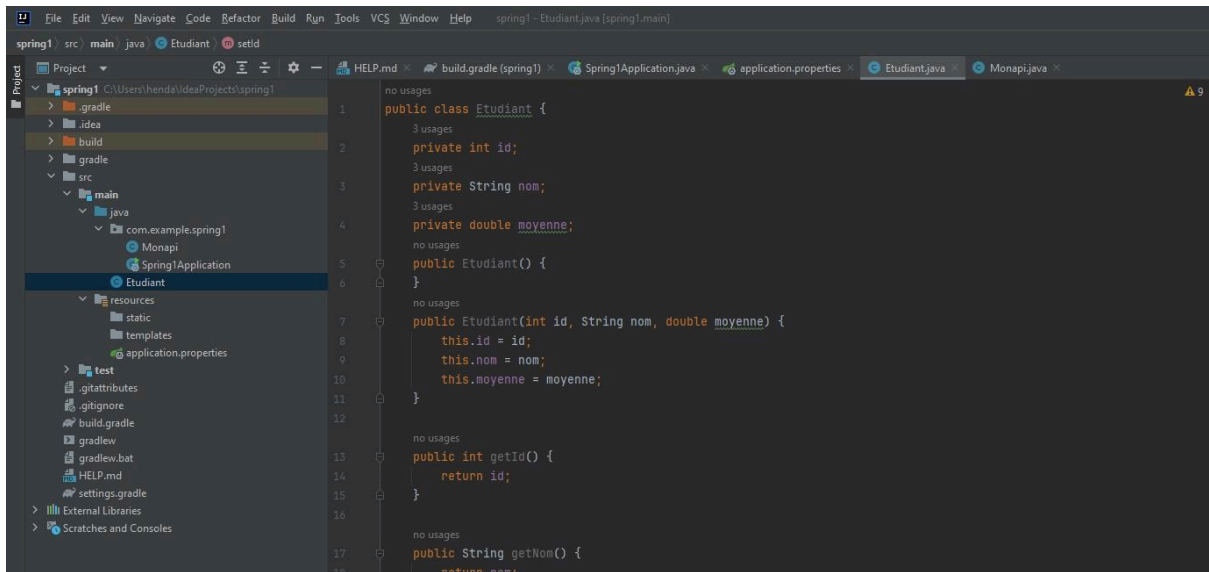
bonjour

configurer tomcat dans application.properties

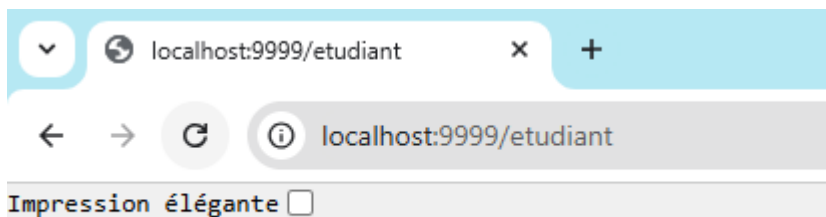


Bonsoir

classe etudiant:



api:



```
{ "id":1, "nom": "A", "moyenne":19.0 }
```

un objet sous format JSON.

Le service web SOAP utilise le format XML.

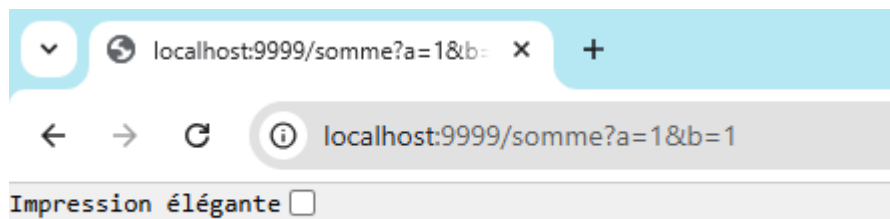
The screenshot shows an IDE with a project named 'spring1'. The project structure on the left includes 'src/main/java/com/example/spring1' with classes 'Etudiant', 'Monapi', and 'Spring1Application'. The 'resources' folder contains 'static', 'templates', and 'application.properties'. The 'test' folder contains various files like '.gitattributes', '.gitignore', 'build.gradle', 'gradlew', 'gradlew.bat', 'HELP.md', and 'settings.gradle'. The main editor displays the 'Monapi' class, which is a REST controller with the following methods:

```
@RestController
public class Monapi {
    @GetMapping(value="/b")
    public String bonjour()
    {
        return "bonjour";
    }

    @GetMapping(value="/bn")
    public String bonsoir()
    {
        return "Bonsoir";
    }

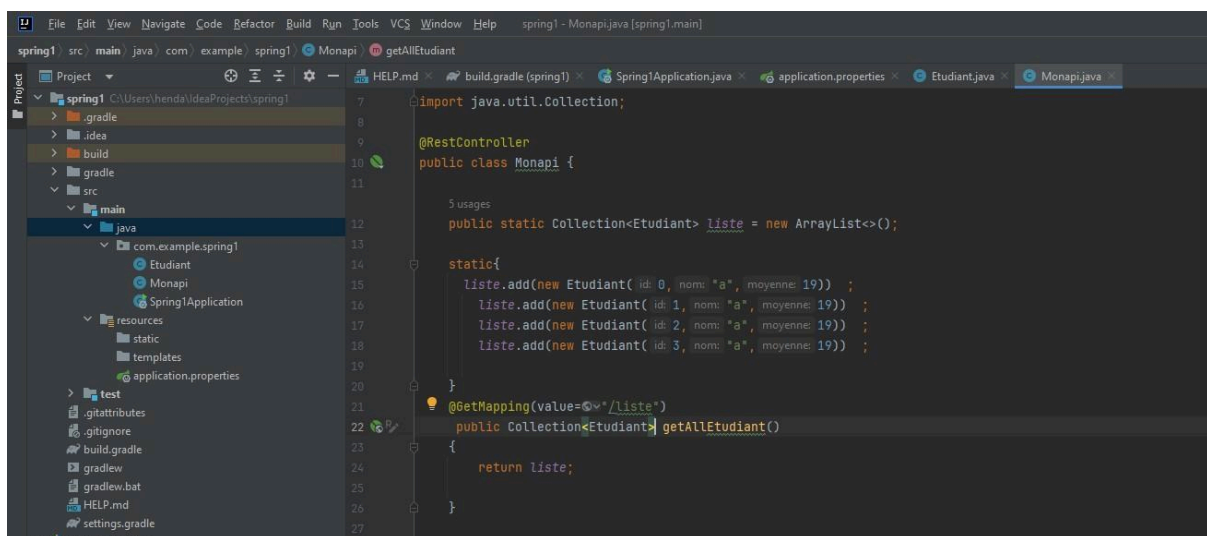
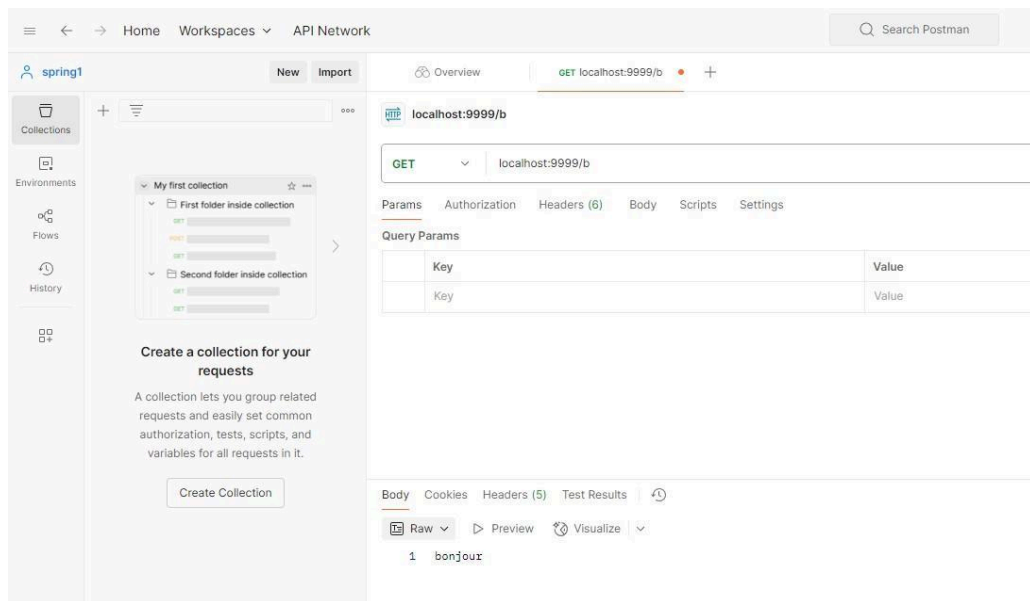
    @GetMapping(value="/etudiant")
    public Etudiant getEtudiant(){
        return new Etudiant( id: 1, nom: "A", moyenne: 19);
    }

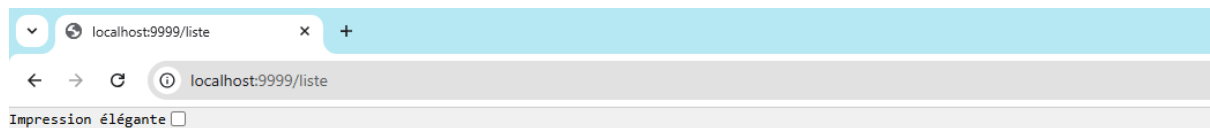
    @GetMapping(value="/somme")
    public double somme(double a, double b)
    { return a+b; }
}
```



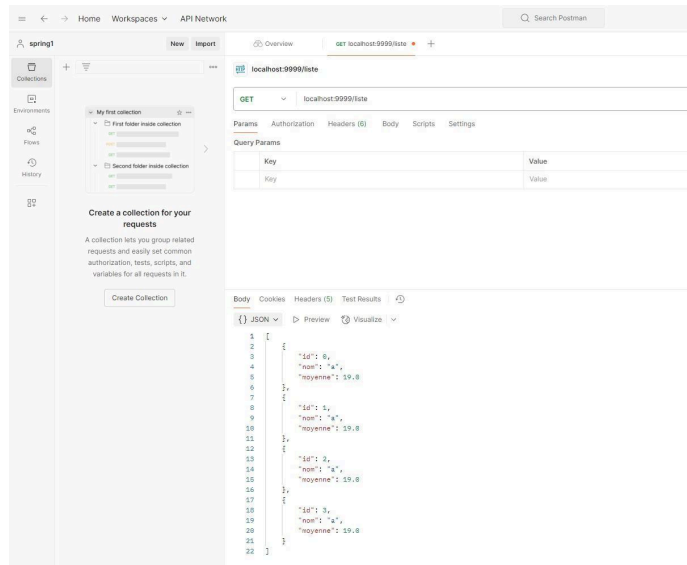
2.0

c postman:

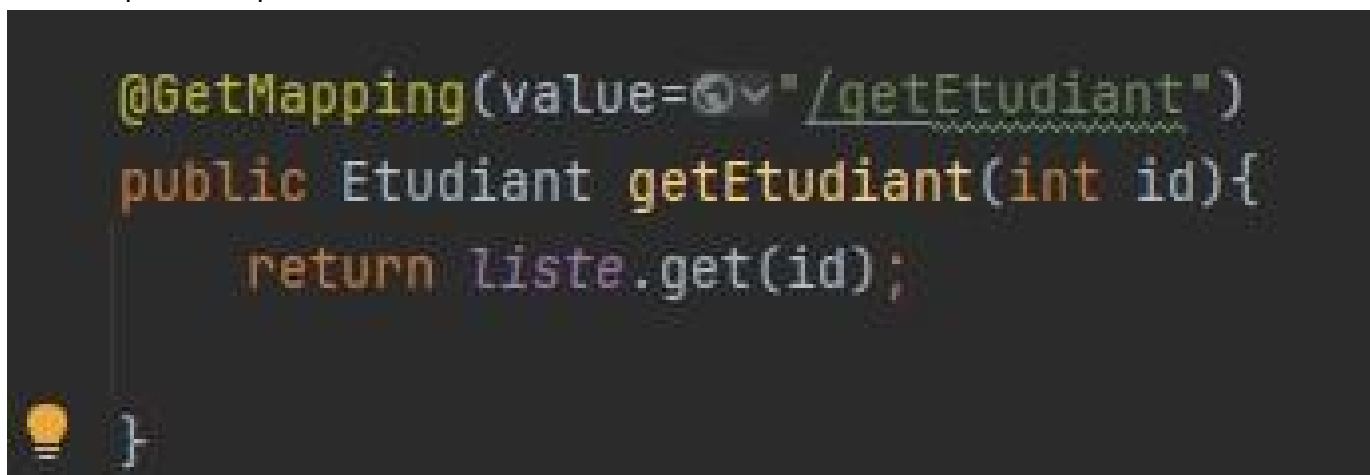


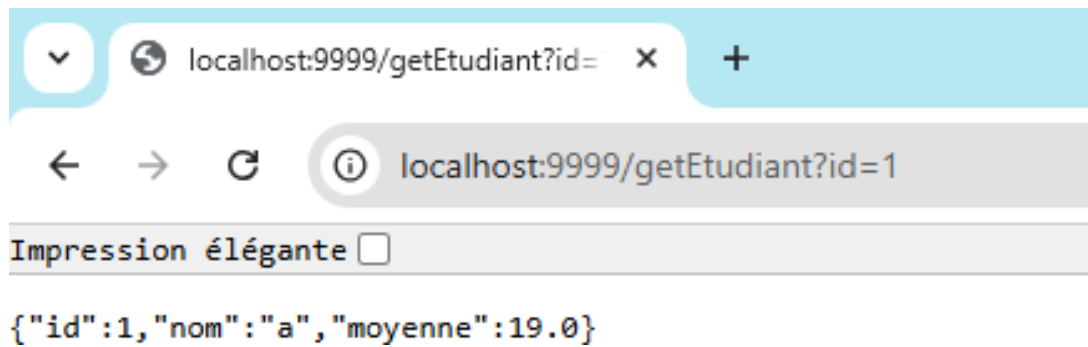


```
[{"id":0,"nom":"a","moyenne":19.0}, {"id":1,"nom":"a","moyenne":19.0}, {"id":2,"nom":"a","moyenne":19.0}, {"id":3,"nom":"a","moyenne":19.0}]
```

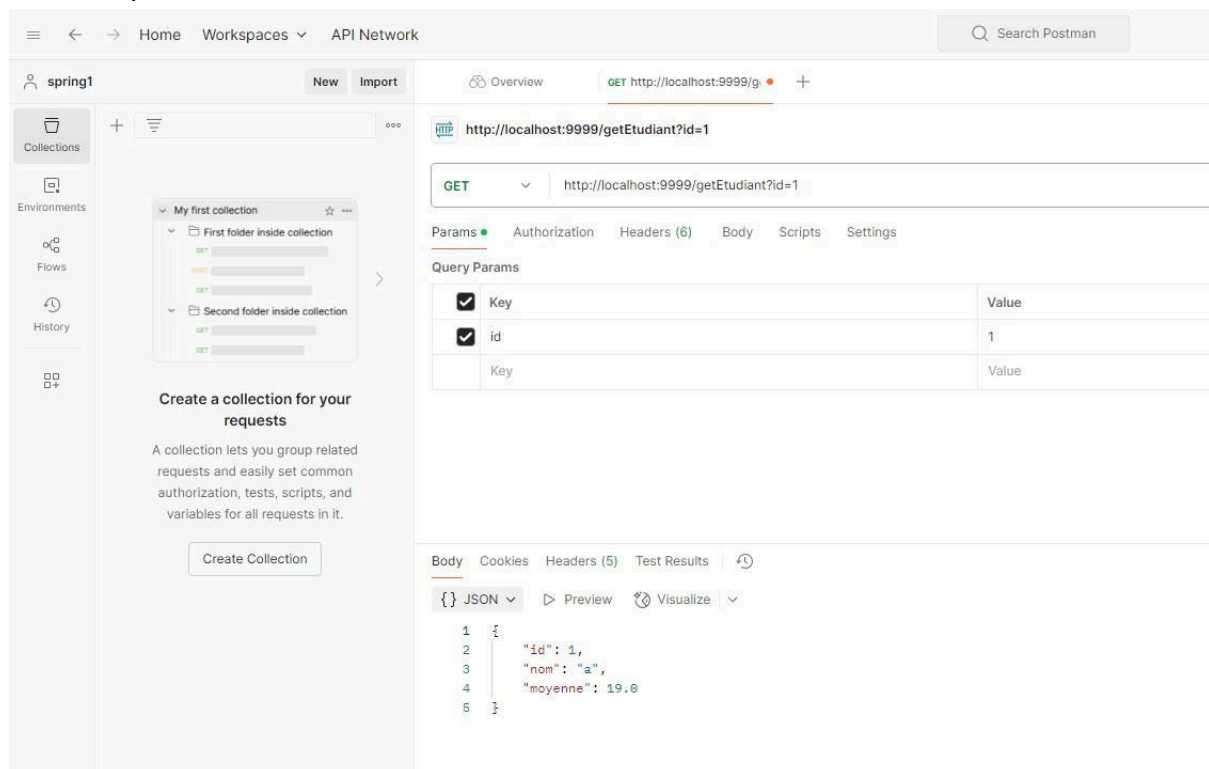


méthode pour récupérer un étudiant avec son id





test avec postman:



```

@PostMapping(value=@RequestMapping("addetudiant"))
public Etudiant addEtudiant(Etudiant etudiant)
{
    liste.add(etudiant);
    return etudiant;
}

```

ajoutetudiant

Postman interface showing a POST request to `http://localhost:9999/addetudiant?id=5&nom=zefezfze&moyenne=20`. The request parameters are:

Key	Value
id	5
nom	zefezfze
moyenne	20

The response body is a JSON object:

```

{
  "id": 5,
  "nom": "zefezfze",
  "moyenne": 20.0
}

```

Browser response showing the JSON array:

```

[{"id":0,"nom":"a","moyenne":19.0}, {"id":1,"nom":"a","moyenne":19.0}, {"id":2,"nom":"a","moyenne":19.0}, {"id":3,"nom":"a","moyenne":19.0}, {"id":5,"nom":"zefezfze","moyenne":20.0}]

```

```

@DeleteMapping(value=@RequestMapping("/delete"))
public void supprimerEtudiant(int id)
{
    liste.remove(id);
}

```

supprimer un etudiant

⌵

← → Home Workspaces ▾ API Network

🔍 Search

👤 spring1

New Import

🗑 Collections

🏠 Environments

📄 Flows

🕒 History

🧩

+ 📄 ⋮ ⋮⋮

My first collection ☆ ⋮⋮

First folder inside collection

POST GET

Second folder inside collection

GET

Create a collection for your requests

A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

🔗 Overview

DEL http://localhost:9999/delete?id=1

+

🔗 http://localhost:9999/delete?id=1

DELETE ▾ http://localhost:9999/delete?id=1

Params Authorization Headers (6) Body Scripts Settings

Query Params

☒	Key	Value
☒	id	1
	Key	Value

Body Cookies Headers (4) Test Results 🕒

📄 Raw ▾ ▶ Preview 🖼 Visualize ▾

1

methode modifier

```
@PutMapping(value="/modifier")
public void modifierEtudiant(int id, String nom) {
    liste.get(id).setNom(nom);
}
```

The screenshot displays the Postman API client interface. The top navigation bar includes 'Home', 'Workspaces', and 'API Network'. The left sidebar shows a 'Collections' panel with a tree view of 'My first collection' containing two folders: 'First folder inside collection' and 'Second folder inside collection'. Below this is a 'Create a collection for your requests' button. The main workspace shows a PUT request to 'http://localhost:9999/modifier?id=2&nom=anis'. The 'Query Params' tab is active, showing a table with three parameters: 'id' with value '2' and 'nom' with value 'anis'. The 'Body' tab is also visible at the bottom.

spring1 New Import Overview PUT http://localhost:9999/modifier?id=2&nom=anis

PUT http://localhost:9999/modifier?id=2&nom=anis

Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value
id	2
nom	anis
Key	Value

Body Cookies Headers (4) Test Results

Raw Preview Visualize

1

```

@SpringBootApplication
public class H2demoApplication {

    public static void main(String[] args) { SpringApplication.run(H2demoApplication.class, args); }

    @Bean
    CommandLineRunner runner(AdherentRepository repository){
        return args -> {
            repository.save(new Adherent( id: null, nom: "a", ville: "b", age: 29));
            repository.save(new Adherent( id: null, nom: "a", ville: "b", age: 29));
            repository.save(new Adherent( id: null, nom: "a", ville: "b", age: 29));
            repository.save(new Adherent( id: null, nom: "a", ville: "b", age: 29));
        }
    }
}

```

injection de dependance

```
@Id
@GeneratedValue(strategy = GenerationType.AUTO)
private Long id;
3 usages
private String nom;
3 usages
private String ville;
3 usages
private int age;

public Adherent() {
}
```

elp H2demo - application.properties [H2demo.main]

HELP.md × H2demoApplication.java × application.properties × Adherent.java × AdherentRepository

```
spring.application.name=H2demo
spring.datasource.url=jdbc:h2:mem:adherent
spring.h2.console.enabled=true
```

H2 Console

localhost:9191/h2-console/login.do?sessionId=6050080f0d492ed794369002cc2848

Auto commit

Max rows: 1000

Auto complete

Off

Auto select

On

jdbc:h2:mem:adherent

ADHERENT

INFORMATION_SCHEMA

Sequences

Users

H2 2.3.232 (2024-08-11)

Run

Run Selected

Auto complete

Clear

SQL statement:

Important Commands

	Displays this Help Page
	Shows the Command History
	Executes the current SQL statement
	Executes the SQL statement defined by the text selection
	Auto complete
	Disconnects from the database

Sample SQL Script

Delete the table if it exists	DROP TABLE IF EXISTS TEST;
Create a new table with ID and NAME columns	CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255));
Add a new row	INSERT INTO TEST VALUES(1, 'Hello');
Add another row	INSERT INTO TEST VALUES(2, 'World');
Query the table	SELECT * FROM TEST ORDER BY ID;
Change data in a row	UPDATE TEST SET NAME='Hi' WHERE ID=1;
Remove a row	DELETE FROM TEST WHERE ID=2;
Help	HELP ...

Adding Database Drivers

Additional database drivers can be registered by adding the Jar file location of the driver to the environment variables H2DRIVERS or CLASSPATH. Example (Windows): to add the database driver library C:\Programs\h2\lib\h2opqdb.jar, set the environment variable H2DRIVERS to C:\Programs\h2\lib\h2opqdb.jar

H2 Console

localhost:9191/h2-console/login.do?jsessionid=6050080f0f49f2ed794f369002cc2848

Auto commit ☒ Max rows: 1000 Auto complete Off Auto select On

jdbc:h2:mem:adherent

- ADHERENT
- INFORMATION_SCHEMA
- Sequences
- Users

H2 2.3.232 (2024-08-11)

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM ADHERENT

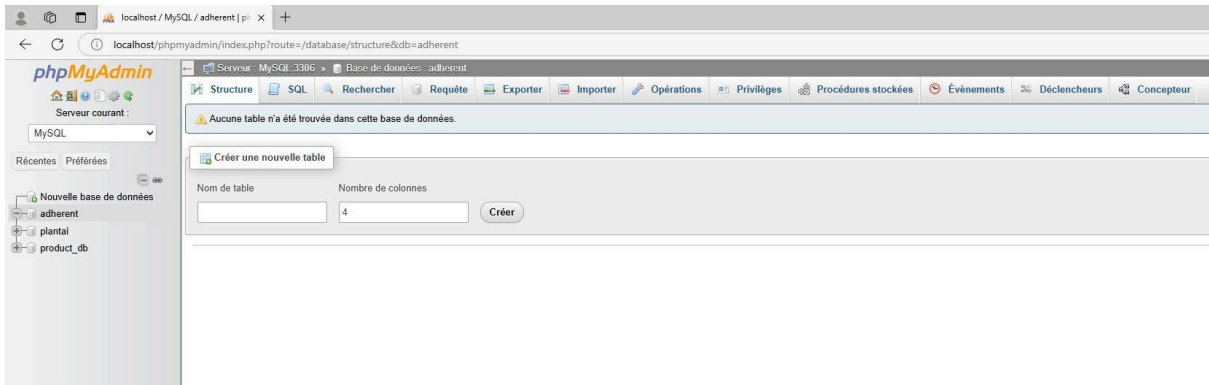
SELECT * FROM ADHERENT;

AGE	ID	NOM	VILLE
29	1	a	b
29	2	a	b
29	3	a	b
29	4	a	b

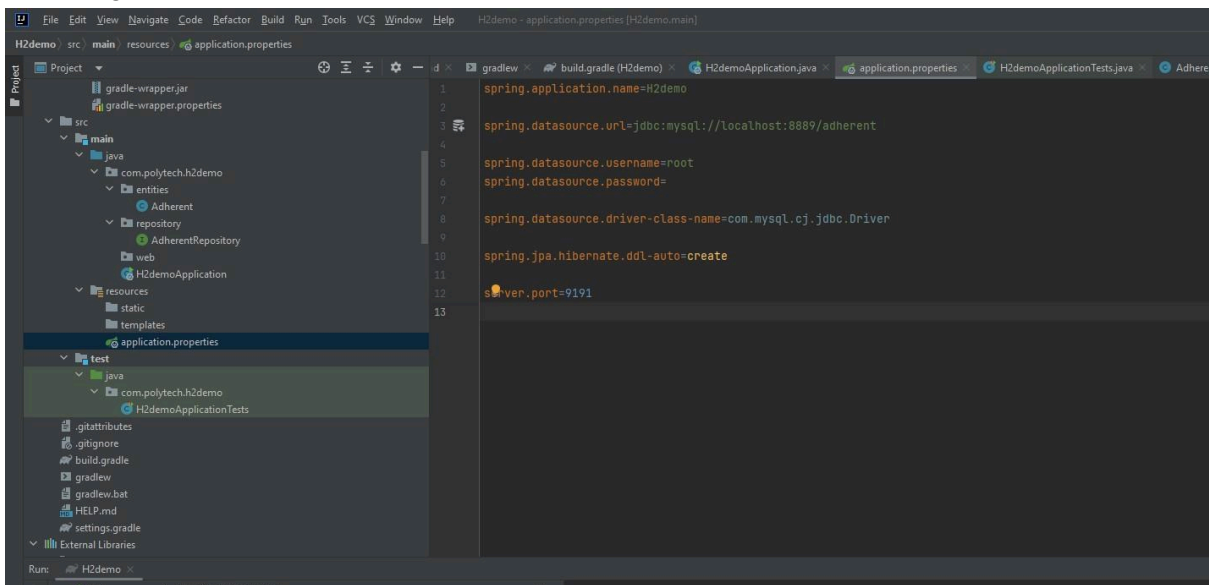
(4 rows, 3 ms)

Edit

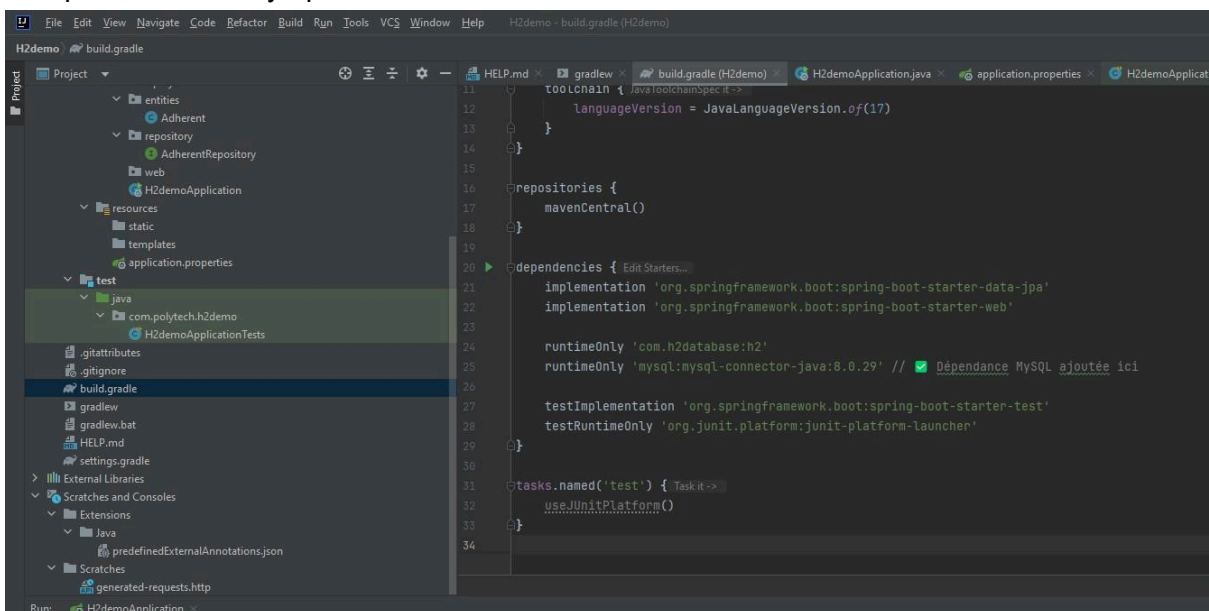
une interface qui hérite de `JpaRepository`.



les configurations sur application.properties



la dépendance de mysql:



To see a list of available tasks, run `gradle tasks`

To see more detail about a task, run `gradle help --task <task>`

To see a list of command-line options, run `gradle --help`

For more detail on using Gradle, see https://docs.gradle.org/8.13/userguide/command_line_interface.html

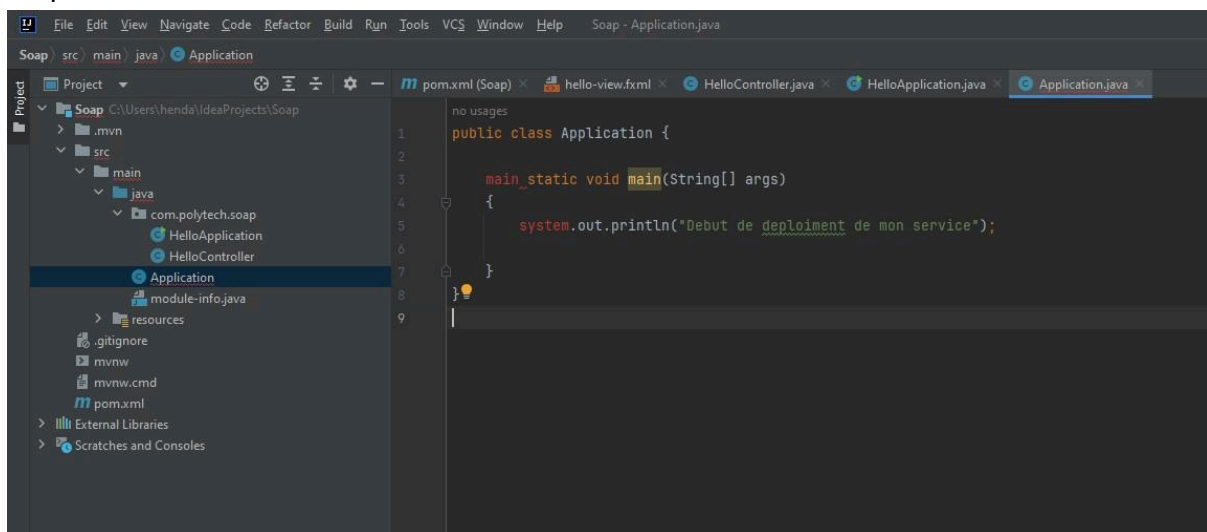
For troubleshooting, visit <https://help.gradle.org>

BUILD SUCCESSFUL in 10s

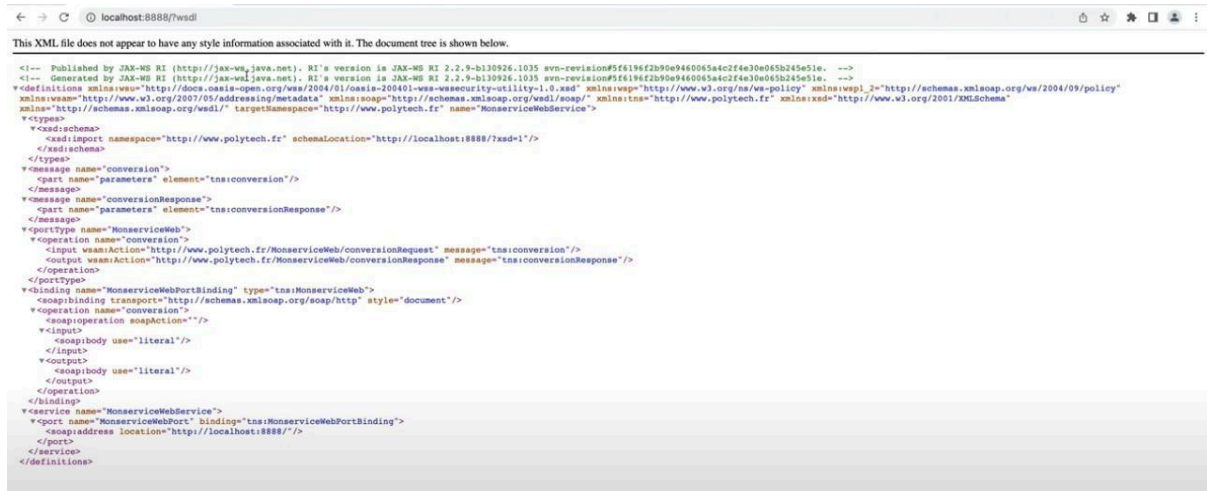
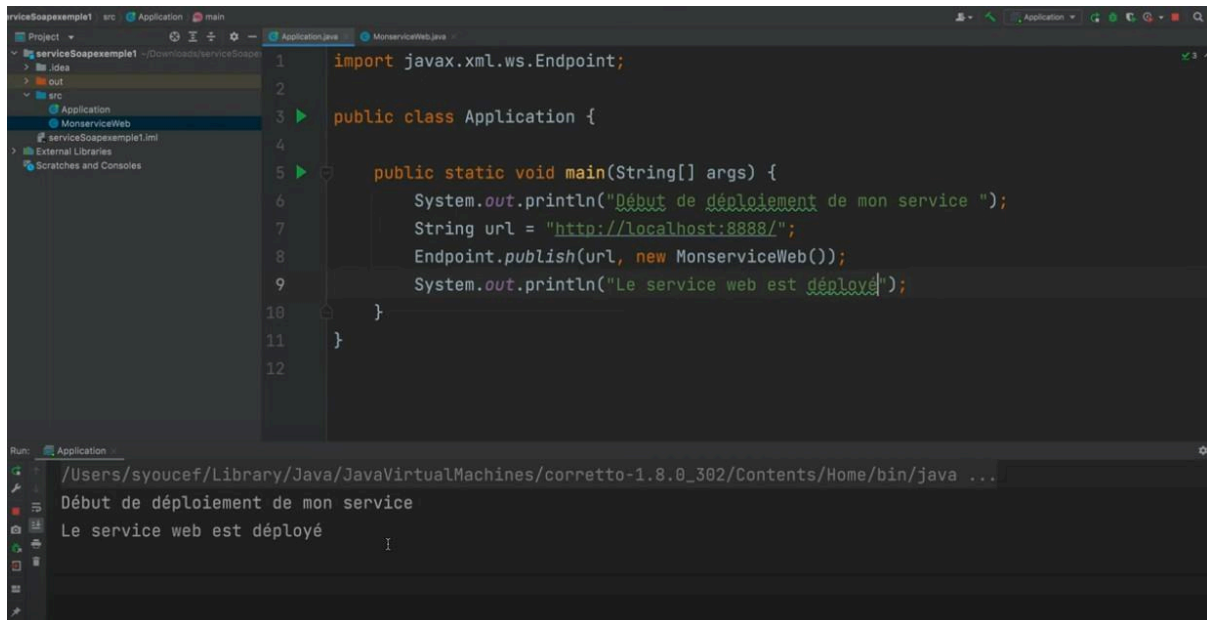
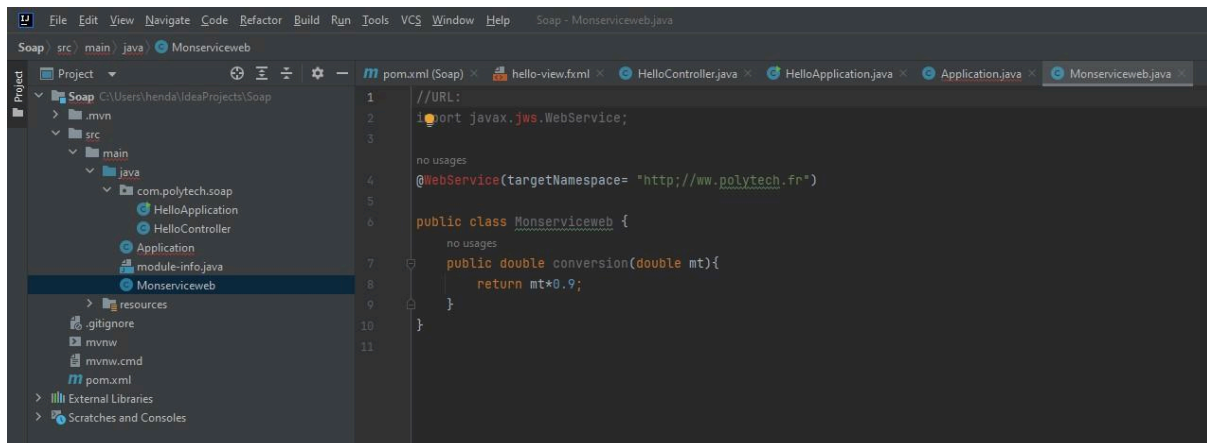
1 actionable task: 1 executed

3:02:39 PM: Execution finished.

soap



URN + URL = URI



Le fichier **WSDL** décrit le service `MonServiceWeb` avec l'opération `conversion`. Le schéma XSD des types est importé depuis

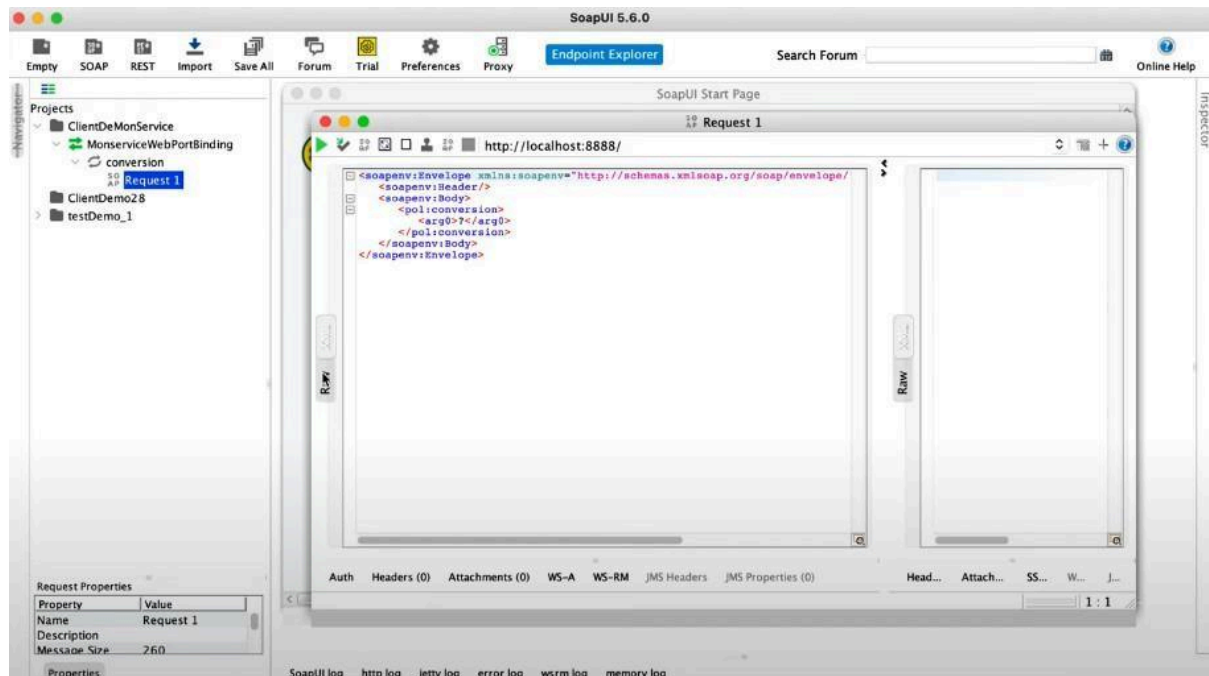
le fichier **XSD** décrit la structure des messages du service SOAP.

L'opération `conversion` prend un **double** (`arg0`) en entrée.

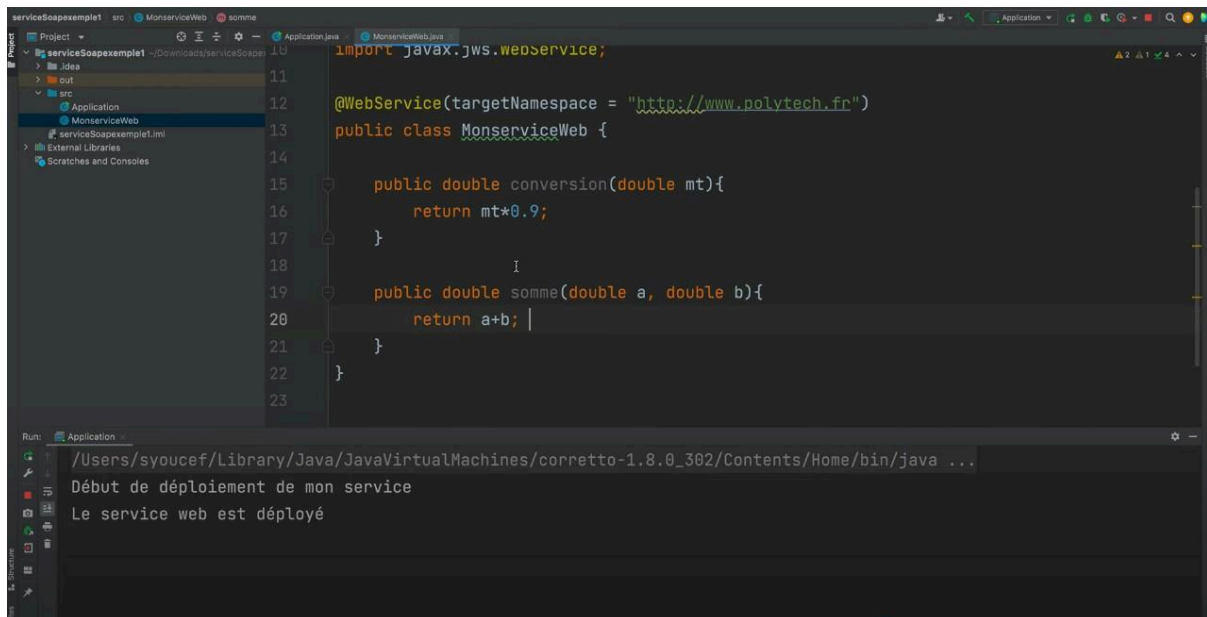
Elle retourne aussi un **double** (`return`) en sortie.

Ce schéma est utilisé par le WSDL pour définir les types XML échangés.

Le proxy transforme les objets Java en XML pour l'échange (sérialisation) et les reconstitue à la réception (désérialisation), assurant ainsi l'interopérabilité entre applications.



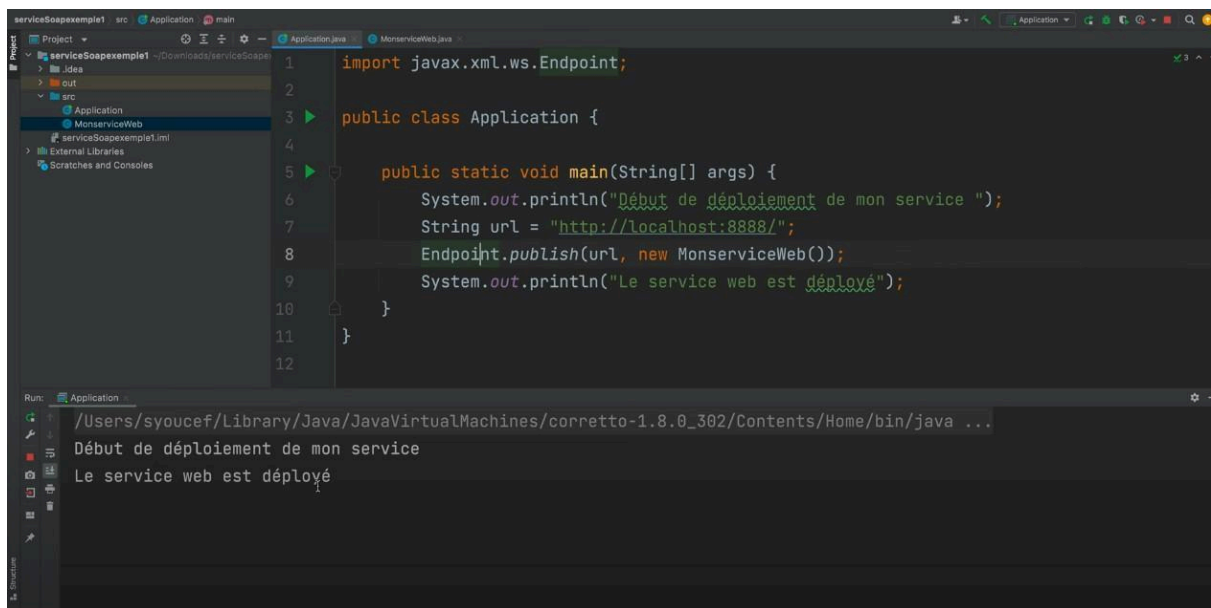
double en entrée, que le service va traiter et te retourner en réponse.



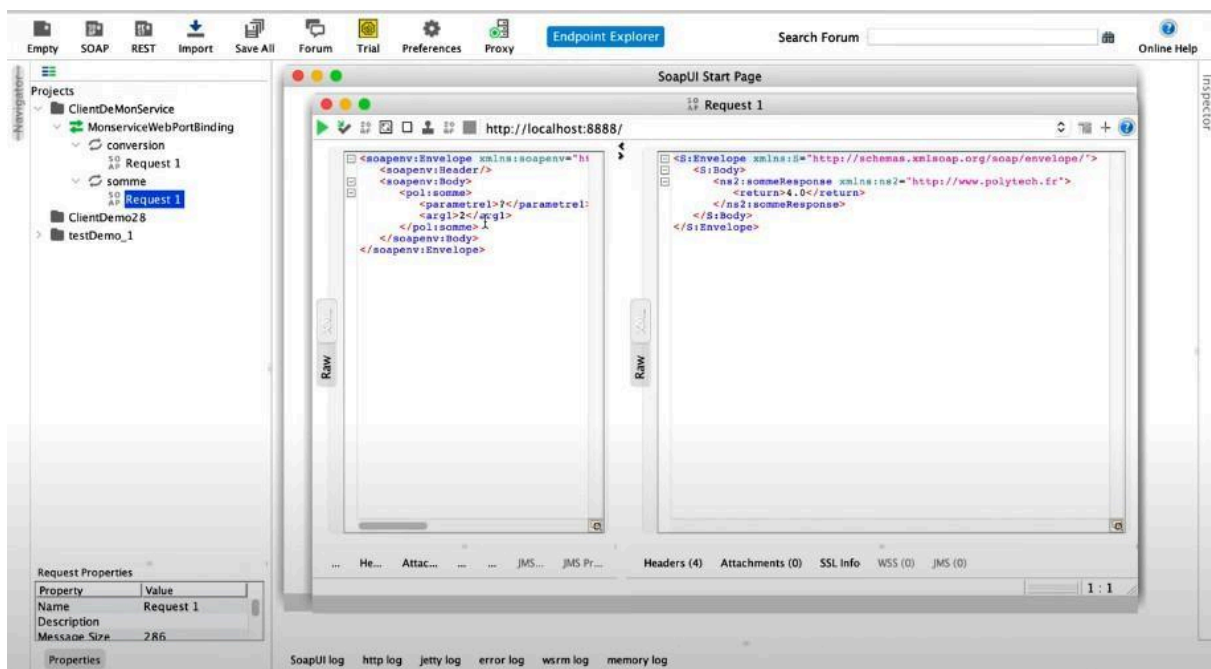
deux opérations dans ton service SOAP :

- conversion(arg0: double) → double
- somme(arg0: double, arg1: double) → double

Ce fichier XSD définit la **structure XML des messages** échangés pour chaque opération.



Service SOAP JAX-WS publié à <http://localhost:8888/>



L'échange SOAP fonctionne bien entre client et service.

```

Application.java  MonServiceWeb.java
1  import javax.xml.ws.WebParam;
2  import javax.xml.ws.WebService;
3
4  @WebService(targetNamespace = "http://www.polytech.fr")
5  public class MonServiceWeb {
6
7      @WebMethod(operationName = "convertir")
8      public double conversion(double mt){
9          return mt*0.9;
10     }
11
12     public double somme(@WebParam(name = "parametre1") double a, double b){

```

@WebService définit la classe comme service SOAP.

@WebMethod permet de personnaliser le nom de l'opération exposée.

@WebParam permet de renommer les paramètres dans le message SOAP pour plus de clarté côté client.

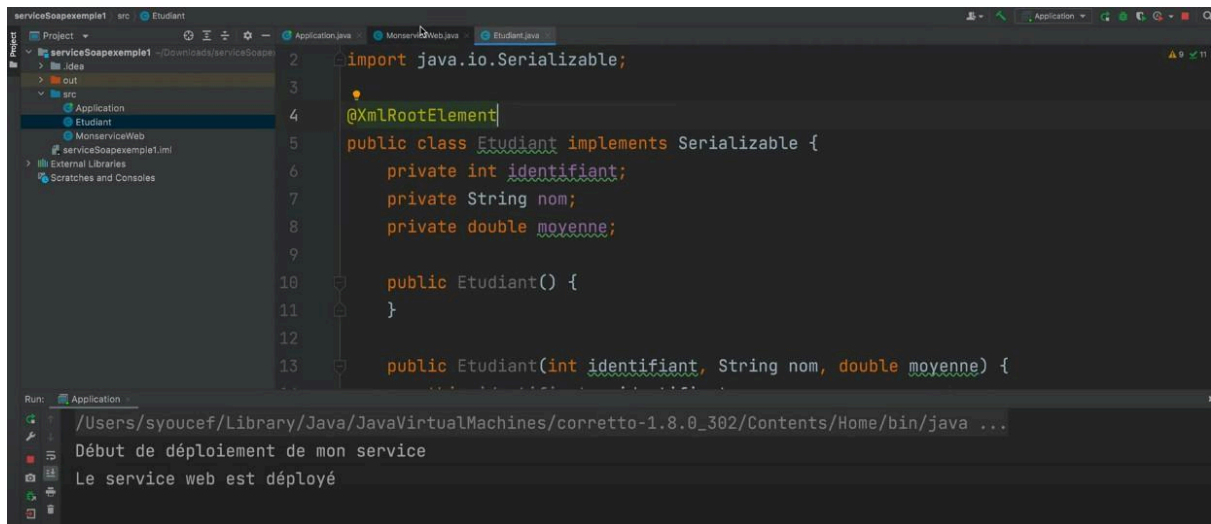
← → ↻ localhost:8888/?wsdl

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```

<!-- Published by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.1035 svn-revision#5f6196f2b90e9460065a4c2f4e30e065b245e51e. -->
<!-- Generated by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.1035 svn-revision#5f6196f2b90e9460065a4c2f4e30e065b245e51e. -->
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<definitions xmlns:wsam="http://schemas.xmlsoap.org/ws/2004/05/addressing/extension" xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:tns="http://www.polytech.fr" targetNamespace="http://www.polytech.fr" name="MonServiceWeb">
  <types>
    <xsd:schema>
      <xsd:import namespace="http://www.polytech.fr" schemaLocation="http://localhost:8888/?xsd=1"/>
    </xsd:schema>
  </types>
  <message name="convertir">
    <part name="parameters" element="tns:convertir"/>
  </message>
  <message name="convertirResponse">
    <part name="parameters" element="tns:convertirResponse"/>
  </message>
  <message name="somme">
    <part name="parameters" element="tns:somme"/>
  </message>
  <message name="sommeResponse">
    <part name="parameters" element="tns:sommeResponse"/>
  </message>
  <portType name="MonServiceWeb">
    <operation name="convertir">
      <input wsam:Action="http://www.polytech.fr/MonServiceWeb/convertirRequest" message="tns:convertir"/>
      <output wsam:Action="http://www.polytech.fr/MonServiceWeb/convertirResponse" message="tns:convertirResponse"/>
    </operation>
    <operation name="somme">
      <input wsam:Action="http://www.polytech.fr/MonServiceWeb/sommeRequest" message="tns:somme"/>
      <output wsam:Action="http://www.polytech.fr/MonServiceWeb/sommeResponse" message="tns:sommeResponse"/>
    </operation>
  </portType>
  <binding name="MonServiceWebPortBinding" type="tns:MonServiceWeb">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
    <operation name="convertir">
      <soap:operation soapAction="">
        <input>
          <soap:body use="literal"/>
        </input>
        <output>
          <soap:body use="literal"/>
        </output>
      </operation>
    <operation name="somme">
      <soap:operation soapAction="">
        <input>
          <soap:body use="literal"/>
        </input>
        <output>
          <soap:body use="literal"/>
        </output>
      </operation>
    </binding>
  </definitions>

```

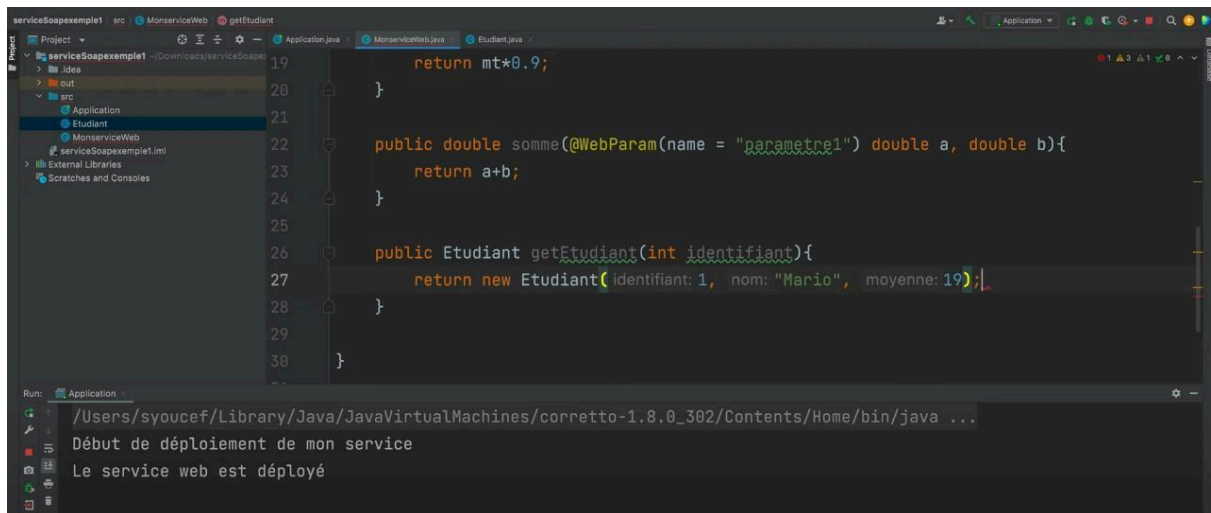


The screenshot shows an IDE with a project named 'serviceSoapexemple1'. The 'src' folder contains 'Application', 'Etudiant', and 'MonServiceWeb'. The 'Etudiant.java' file is open, showing the following code:

```
2 import java.io.Serializable;
3
4 @XmlRootElement
5 public class Etudiant implements Serializable {
6     private int identifiant;
7     private String nom;
8     private double moyenne;
9
10    public Etudiant() {
11    }
12
13    public Etudiant(int identifiant, String nom, double moyenne) {
```

The console output at the bottom shows the command: `/Users/syoucef/Library/Java/JavaVirtualMachines/corretto-1.8.0_302/Contents/Home/bin/java ...` and the messages: `Début de déploiement de mon service` and `Le service web est déployé`.

JAXB va **convertir l'objet en XML** lors de l'appel du service.



The screenshot shows the same IDE with the 'MonServiceWeb.java' file open. The code is as follows:

```
19 return mt*0.9;
20 }
21
22 public double somme(@WebParam(name = "parametre1") double a, double b){
23     return a+b;
24 }
25
26 public Etudiant getEtudiant(int identifiant){
27     return new Etudiant(identifiant: 1, nom: "Mario", moyenne: 19);
28 }
29
30 }
```

The console output at the bottom is identical to the first screenshot, showing the command: `/Users/syoucef/Library/Java/JavaVirtualMachines/corretto-1.8.0_302/Contents/Home/bin/java ...` and the messages: `Début de déploiement de mon service` and `Le service web est déployé`.

